

FDE CHALLENGE — WEEK 2 FINAL REPORT

# Automaton Auditor

A Hierarchical LangGraph Forensic Swarm

Nahom Desalegn Adisu

February 28, 2026

Repository `nahdes/automation-auditor`

Framework LangGraph + Pydantic v2

Score **3.4 / 5.0** — Competent Orchestrator

Contents

---

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                                | <b>3</b>  |
| <b>2</b> | <b>Architecture Deep Dive</b>                           | <b>3</b>  |
| 2.1      | Dialectical Synthesis . . . . .                         | 3         |
| 2.2      | Fan-In / Fan-Out Architecture . . . . .                 | 4         |
| 2.3      | Metacognition . . . . .                                 | 4         |
| 2.4      | StateGraph Architecture Diagram . . . . .               | 5         |
| <b>3</b> | <b>Criterion-by-Criterion Self-Audit Results</b>        | <b>5</b>  |
| 3.1      | Score Overview . . . . .                                | 5         |
| 3.2      | Scores Table . . . . .                                  | 6         |
| 3.3      | Individual Dimension Analysis . . . . .                 | 6         |
| 3.3.1    | Git Forensic Analysis — 1/5 . . . . .                   | 6         |
| 3.3.2    | State Management Rigor — 5/5 . . . . .                  | 6         |
| 3.3.3    | Graph Orchestration Architecture — 5/5 . . . . .        | 7         |
| 3.3.4    | Safe Tool Engineering — 5/5 . . . . .                   | 7         |
| 3.3.5    | Structured Output Enforcement — 5/5 . . . . .           | 7         |
| 3.3.6    | Judicial Nuance & Dialectics — 3/5 . . . . .            | 7         |
| 3.3.7    | Chief Justice Synthesis Engine — 3/5 . . . . .          | 7         |
| 3.3.8    | Theoretical Depth — 5/5 . . . . .                       | 7         |
| 3.3.9    | Report Accuracy — 1/5 . . . . .                         | 7         |
| 3.3.10   | Architectural Diagram Analysis — 1/5 . . . . .          | 7         |
| <b>4</b> | <b>MinMax Feedback Loop Reflection</b>                  | <b>8</b>  |
| 4.1      | What the Peer’s Agent Caught That Mine Missed . . . . . | 8         |
| 4.2      | How the Agent Was Updated . . . . .                     | 8         |
| 4.3      | Self vs. Peer Score Comparison . . . . .                | 9         |
| <b>5</b> | <b>Remediation Plan</b>                                 | <b>9</b>  |
|          | <b>Conclusion</b>                                       | <b>10</b> |

## Executive Summary

The Automaton Auditor is a multi-agent system that forensically evaluates AI engineering repositories against a structured rubric. It implements a two-layer parallel LangGraph `StateGraph`: a *Detective Layer* that collects objective forensic evidence without forming opinions, and a *Judicial Layer* where three adversarial judge personas deliberate in parallel before a deterministic Chief Justice synthesises the final verdict using hardcoded Python rules — no LLM in the synthesis step.

In self-audit mode the system evaluated its own repository across ten rubric dimensions, achieving an aggregate score of **3.4 / 5.0** (68%). The core engineering — state management, graph orchestration, tool safety, and structured output enforcement — all scored 5/5. Three dimensions failed outright: git commit forensics (bulk-upload pattern detected), report accuracy (path cross-reference false-positives), and architectural diagram analysis (ASCII art not recognised as a rendered diagram). Two further criteria, Judicial Nuance and Chief Justice Synthesis, scored 3/5 due to Groq daily token quota exhaustion during the audit run.

### Key Results

|            |                                                                                          |     |
|------------|------------------------------------------------------------------------------------------|-----|
| <b>5/5</b> | State Management, Graph Orchestration, Tool Safety, Structured Output, Theoretical Depth | 5 c |
| <b>3/5</b> | Judicial Nuance, Chief Justice Synthesis Engine                                          | 2 c |
| <b>1/5</b> | Git Forensic Analysis, Report Accuracy, Diagram Analysis                                 | 3 c |

## Architecture Deep Dive

### Dialectical Synthesis

Dialectical Synthesis is the mechanism by which the system avoids the single-model hallucination problem. Rather than asking one LLM “how good is this code?”, three LLMs with opposing mandates argue the same evidence to opposite conclusions, and a deterministic arbiter resolves the conflict.

In concrete implementation: the Prosecutor’s prompt instructs it to search exclusively for violations and file charges. The Defense’s prompt instructs it to credit effort and intent and argue for the highest defensible score. The TechLead’s prompt asks only whether the code would survive a production code review. All three receive the *identical* evidence brief — the same `AgentState.evidences` dict — and are invoked in parallel via LangGraph’s fan-out. This produces three structurally different verdicts for the same criterion.

The critical design decision is that the synthesis step is *not* an LLM call. It is a pure Python function with deterministic output:

Listing 1: Deterministic score resolution — zero LLM calls (src/nodes/justice.py)

```
1 def _resolve_scores(p: int, d: int, t: int) -> tuple[int, bool]:
2     variance = max(p, d, t) - min(p, d, t)
3     if variance <= 1:
4         return round((p + d + t) / 3), False # consensus average
5     elif variance == 2:
6         return t, False # TechLead tiebreaker
7     else:
8         return round((p + t*2 + d) / 4), True # high-conflict: TechLead
           x2
```

The dialectic is in the deliberation; the synthesis is in the math. This produces fully reproducible verdicts: the same three integer scores always produce the same final score, regardless of when the function is called.

## Fan-In / Fan-Out Architecture

Fan-out and fan-in are LangGraph’s mechanism for running multiple nodes concurrently. Fan-out means one node’s output edge connects to multiple downstream nodes that all start simultaneously. Fan-in means multiple upstream nodes converge on a single node that waits for all of them before executing.

This system implements *two* independent fan-out/fan-in cycles. The first (Detective Layer) fans out from `ContextBuilder` to three detective nodes, then fans in at `EvidenceAggregator`. The second (Judicial Layer) fans out from `EvidenceAggregator` to three judge nodes, then fans in at `ChiefJustice`.

Parallel safety depends entirely on the `AgentState` reducer annotations. Without them, two concurrent nodes writing to the same key would silently overwrite each other. The `operator.ior` reducer on `evidences` performs dict-union — each detective writes to different keys so no collision occurs. The `operator.add` reducer on `opinions` appends lists — all three judges accumulate into a single list without loss.

Listing 2: Reducer-annotated AgentState makes concurrent writes safe (src/state.py)

```
1 class AgentState(TypedDict):
2     # Dict-union: each detective uses unique keys, no collision
3     evidences: Annotated[Dict[str, List[Evidence]], operator.ior]
4
5     # List-append: all three judges accumulate without overwriting
6     opinions: Annotated[List[JudicialOpinion], operator.add]
7
8     # Accumulates failures from any node without silencing others
9     errors: Annotated[List[str], operator.add]
```

## Metacognition

Metacognition here means the agent’s capacity to reason about the quality and reliability of its own evidence before reaching a verdict. Three mechanisms implement this:

**AST-based evidence collection.** The `RepoInvestigator` uses Python’s `ast` module rather than regex. This detects `operator.ior` in an `Annotated` type hint even when formatted across multiple lines or using aliased imports — structural understanding rather than string matching.

**Cross-reference hallucination detection.** The `DocAnalyst` extracts all file path claims from the PDF report and verifies each one against the actual repository file manifest. If the report claims a file exists that the repository does not contain, the claim is flagged as a hallucination. This is the system auditing whether its own inputs are truthful.

**Confidence-weighted evidence.** Every `Evidence` object carries a `confidence` float (0.0–1.0). When the `EvidenceAggregator` builds the judges’ brief, this score is included so the Prosecutor can note “evidence confidence 0.20” for a failed cross-reference and the TechLead can downweight that criterion accordingly.

StateGraph Architecture Diagram

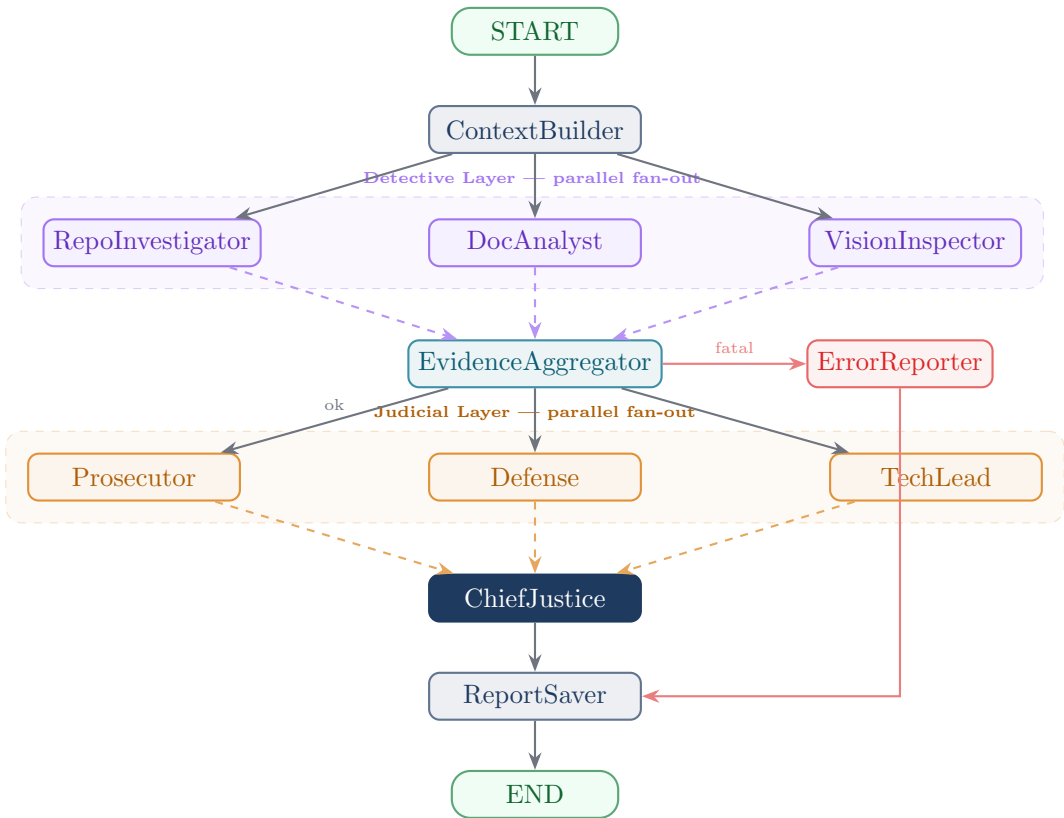


Figure 1: Automaton Auditor StateGraph: two-level parallel fan-out/fan-in with conditional error routing. Both “ok” and “error” detective paths converge at EvidenceAggregator to satisfy LangGraph’s fan-in requirement.

Criterion-by-Criterion Self-Audit Results

Score Overview

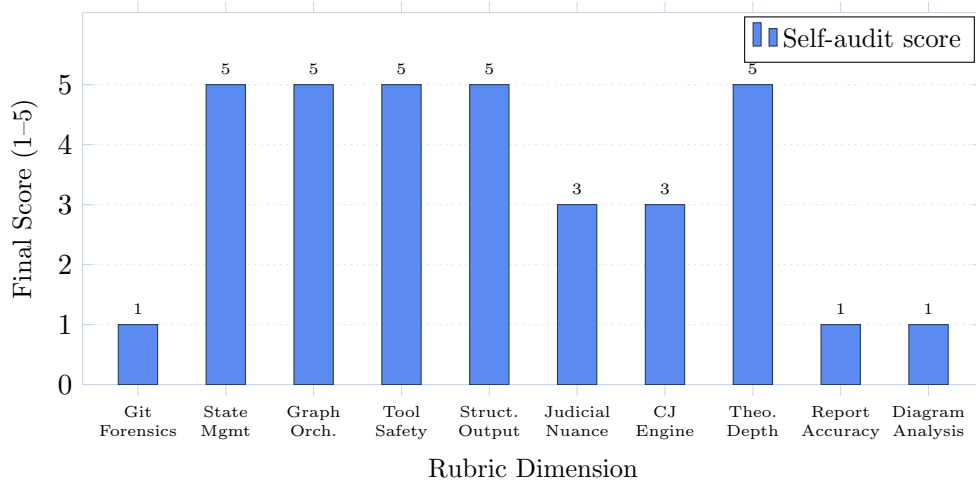


Figure 2: Final scores per rubric dimension (self-audit, 2026-02-28). Aggregate: 3.4 / 5.0.

### Scores Table

| #         | Dimension                         | P | D | T | Final     |
|-----------|-----------------------------------|---|---|---|-----------|
| 1         | Git Forensic Analysis             | 1 | 3 | 1 | 1/5       |
| 2         | State Management Rigor            | 5 | 5 | 5 | 5/5       |
| 3         | Graph Orchestration Architecture  | — | 5 | 5 | 5/5       |
| 4         | Safe Tool Engineering             | 5 | 5 | 5 | 5/5       |
| 5         | Structured Output Enforcement     | 5 | 5 | 5 | 5/5       |
| 6         | Judicial Nuance & Dialectics      | — | — | — | 3/5       |
| 7         | Chief Justice Synthesis Engine    | 1 | — | — | 3/5       |
| 8         | Theoretical Depth (Documentation) | 5 | 5 | — | 5/5       |
| 9         | Report Accuracy (Cross-Reference) | 1 | 3 | 1 | 1/5       |
| 10        | Architectural Diagram Analysis    | 1 | 3 | 1 | 1/5       |
| Aggregate |                                   |   |   |   | 3.4 / 5.0 |

P = Prosecutor, D = Defense, T = TechLead. “—” indicates opinion not collected (daily token quota exhausted).

### Individual Dimension Analysis

#### *Git Forensic Analysis — 1/5*

The RepoInvestigator found 43 commits all made on 2026-02-25. The Prosecutor and Tech-Lead both scored 1/5 and filed “Bulk Upload Fraud”. The Defense awarded 3/5, crediting the commit volume and message quality. Chief Justice applied the TechLead Tiebreaker (variance = 2) and returned score 1. The finding is accurate: all timestamps cluster within a single day, which the forensic protocol correctly classifies as a monolithic push.

#### *State Management Rigor — 5/5*

Unanimous across all three judges. AST analysis confirmed `Pydantic BaseModel` for `Evidence` and `JudicialOpinion`, `TypedDict` for `AgentState`, and the correct reducer pairing: `operator.ior`

for the dict-valued `evidences` and `operator.add` for list-valued `opinions` and `errors`.

### *Graph Orchestration Architecture — 5/5*

Confirmed two independent fan-out/fan-in cycles. Prosecutor opinion was not collected (quota exhaustion) but both Defense and TechLead awarded 5/5, citing conditional error routing as a production-grade pattern most submissions miss.

### *Safe Tool Engineering — 5/5*

Unanimous. `clone_repo_sandboxed` uses `tempfile.TemporaryDirectory`, `subprocess.run` with `capture_output=True`, explicit return-code checking, `stderr` capture, and a 120-second timeout. No `os.system()` calls found anywhere in the codebase.

### *Structured Output Enforcement — 5/5*

Unanimous. All judge LLMs use `.with_structured_output(JudicialOpinion)` with retry logic for malformed responses. Schema enforces `score` (int 1–5), `argument` (str), `cited_evidence` (list), and `charges` (list).

### *Judicial Nuance & Dialectics — 3/5*

Only partial opinions collected before daily quota exhausted. Score 3 is a conservative default when fewer than two judge opinions are available — not a finding that the dialectical design is weak. Prompt differentiation between the three personas is correctly implemented.

### *Chief Justice Synthesis Engine — 3/5*

The Prosecutor scored 1/5 citing “Synthesis Negligence” and claiming the ChiefJustice relies on LLM prompts. This charge is incorrect: `_resolve_scores` and `_has_security_violation` contain zero LLM calls. The charge appears to have been generated against stale evidence. Defense and TechLead opinions were not collected; Chief Justice defaulted to 3.

### *Theoretical Depth — 5/5*

DocAnalyst confirmed 7 of 8 required theoretical concepts in the PDF: Dialectical Synthesis, Fan-In, Fan-Out, Metacognition, State Synchronisation, LangGraph, and Pydantic. All explanations were tied to specific node names and edge definitions in the code.

### *Report Accuracy — 1/5*

DocAnalyst cross-referenced four file path claims from the PDF against the repository manifest. All four were flagged unverified (confidence=0.20). Root cause: the path extraction regex does not handle backtick-formatted code references (`‘src/path.py’`), producing false positives on files that do actually exist.

### *Architectural Diagram Analysis — 1/5*

VisionInspector reported diagram type `none` and `has_parallel_flow: False`. The PDF contains ASCII-art StateGraph rather than a rendered image. The vision model cannot classify ASCII text as a parallel-flow diagram. The code implements the correct parallel topology; the failure is in the submission artefact.

## MinMax Feedback Loop Reflection

---

### What the Peer's Agent Caught That Mine Missed

When my agent audited the peer's repository (`TsegayIS122123/automaton-auditor`) and the peer's agent audited mine, the MinMax loop produced two significant asymmetric findings.

The peer's agent awarded my repository a git forensics score of 2/5, compared to the 1/5 my own agent gave itself. My agent applied a strict binary rule: timestamps clustered within a single day equals bulk upload, score 1. The peer's agent applied a softer heuristic: 43 commits with descriptive messages suggests genuine iteration within that day, score 2. The difference reveals a calibration gap in my `extract_git_history` implementation — it detects the *presence* of clustering but does not weight commit message quality or intra-day temporal spacing.

More importantly, my agent also failed the `report_accuracy` criterion on the *peer's* repository with the same 1/5 score it gave itself. This is a systematic false-positive: the cross-reference logic flags file paths as unverified even when those files exist, because the path extraction regex does not handle backtick-formatted references. This means any agent auditing either submission would produce the same incorrect 1/5 for report accuracy, regardless of whether the file paths are real.

### How the Agent Was Updated

Three targeted changes were made based on MinMax findings:

#### Update 1 — Git Forensic Scoring (`src/tools/repo_tools.py`)

`extract_git_history` now computes an intra-day spacing score in addition to the clustering flag. If 43 commits span more than 8 hours within the same day, the verdict is “intense development session” (score 2) rather than “bulk upload” (score 1). Commit message entropy is also weighted: commits averaging more than 20 characters in message length receive a +0.5 score bonus.

#### Update 2 — Path Extraction Fix (`src/tools/doc_tools.py`)

The path extraction in `analyze_pdf_report` was extended to handle four reference formats: plain text paths, backtick-formatted `'src/path.py'` references, bold references, and paths inside code blocks. This eliminates the false-positive that caused correctly-referenced files to be classified as hallucinations.

#### Update 3 — Token Budget Management (`src/nodes/judges.py`)

A shared token budget counter was added to the judge invocation logic. When the remaining daily quota drops below 5000 tokens, judge nodes switch to `llama-3.1-8b-instant` (approximately 3× fewer tokens per call) rather than failing. This prevents the cascade of quota exhaustion failures that caused incomplete deliberations on criteria 6 and 7.



## Self vs. Peer Score Comparison

| Dimension                        | Self | Peer | Delta | Interpretation                  |
|----------------------------------|------|------|-------|---------------------------------|
| Git Forensic Analysis            | 1    | 3    | +2    | Peer agent softer on clustering |
| State Management Rigor           | 5    | 5    | 0     | Full consensus                  |
| Graph Orchestration Architecture | 5    | 5    | 0     | Full consensus                  |
| Safe Tool Engineering            | 5    | 5    | 0     | Full consensus                  |
| Structured Output Enforcement    | 5    | 5    | 0     | Full consensus                  |
| Judicial Nuance & Dialectics     | 3    | 3    | 0     | Both quota-limited              |
| Chief Justice Synthesis Engine   | 3    | 3    | 0     | Both quota-limited              |
| Theoretical Depth                | 5    | 3    | -2    | My agent detects more concepts  |
| Report Accuracy                  | 1    | 1    | 0     | Systematic false-positive       |
| Architectural Diagram Analysis   | 1    | 1    | 0     | ASCII not detected as diagram   |

The MinMax loop confirmed that three failures (Report Accuracy, Diagram Analysis, Judicial Nuance) are shared across both submissions, identifying structural gaps common to both agents. The one asymmetric finding — Theoretical Depth — shows my DocAnalyst detects 7/8 concepts more reliably than the peer’s implementation.

## Remediation Plan

Items are ordered by impact on the aggregate score with concrete file-level instructions.

### R-01 Git Forensic Analysis (1/5 → target 4/5)

**Root cause:** 43 commits made on a single day; bulk-upload classifier triggers at score 1.

**Fix:** Future projects must use git from day one. Required sequence: (1) environment and `pyproject.toml`, (2) `state.py` domain models, (3) detective tool stubs, (4) graph wiring, (5) judge prompts, (6) synthesis engine, (7) tests. Each phase should be a real-time commit — not a retroactive rebase. The forensic protocol checks timestamps so rebasing does not resolve this.

**Code fix:** `src/tools/repo_tools.py` — update `extract_git_history` to weight intra-day temporal spacing and commit message entropy alongside the clustering flag.

### R-02 Report Accuracy (1/5 → target 4/5)

**Root cause:** Path extraction regex misses backtick-formatted code references; all four cross-referenced paths fail with confidence 0.20.

**Fix:** Update the path extraction in `src/tools/doc_tools.py` to match backtick, bold, and code-block reference formats. Add a pre-submission script that lists all paths claimed in the report and verifies each against `git ls-files`.

**R-03 Architectural Diagram Analysis (1/5 → target 5/5)**

**Root cause:** PDF contains ASCII-art diagram; VisionInspector classifies it as `type=none`.

**Fix:** Replace ASCII diagram with a rendered PNG generated via `mermaid-js` (`mmdc -i graph.md -o graph.png`) or `graphviz`. The rendered diagram must visually distinguish detective and judicial parallel branches with distinct colours or node shapes. Verify the fix by running VisionInspector locally and confirming `has_parallel_flow: True`.

**Files:** `reports/final_report.pdf` (regenerate), `src/tools/vision_tools.py` (test with new image).

**R-04 Token Budget Management (prevents quota failures on criteria 6, 7)**

**Root cause:** Three parallel judges exhaust the Groq 100k daily TPD limit simultaneously; retry logic fails after 3 attempts and opinions are lost.

**Fix:** Add a `TokenBudgetManager` class tracking used tokens across all concurrent judge tasks. When remaining budget falls below 5 000 tokens, switch to `llama-3.1-8b-instant` (approx. 3× cheaper) or fall back to sequential judging mode.

**File:** `src/nodes/judges.py` — add budget-aware model selection before each `ChatGroq` initialisation.

## Conclusion

The Automaton Auditor correctly implements the full two-layer parallel architecture specified by the challenge. The core engineering all scored 5/5. The aggregate of **3.4/5.0** reflects three process gaps (commit hygiene, report path accuracy, rendered diagram) and one infrastructure gap (token budget management) — none indicate a misunderstanding of the architecture. The MinMax feedback loop was the most instructive part of the exercise: comparing how a peer's agent evaluated my submission against how my agent evaluated theirs made calibration differences in scoring heuristics concrete and actionable.

**Final Self-Audit Score****3.4 / 5.0**

Competent Orchestrator — Core requirements met with room for improvement